

Assignment/Quiz-RNN/LSTM/GRU

Name: Bernard G. Yasay

Doctor of Engineering with Specialization in Computer Engineering

DOES 1C3-DIT/DENG11G1 – Bioinformatics

1. **Modify the provided notebooks** for RNN, LSTM, and GRU, which are available here:
https://drive.google.com/drive/folders/14FmlwldlsdtU_ZUOv4C8ViRuakqfS6jDe?usp=sharing

Add another layer of the same type to each model:

- Add another **RNN layer** to the RNN model.

```
[15]: X1, y1 = make_one_to_one(train_scaled)
      Xv1, yv1 = make_one_to_one(val_scaled)
      print("X shape:", X1.shape, " y shape:", y1.shape)

      model_one_to_one = tf.keras.Sequential([
          tf.keras.layers.Input(shape=(1, N_FEATURES)),
          tf.keras.layers.SimpleRNN(UNITS, return_sequences=True),
          tf.keras.layers.SimpleRNN(UNITS, return_sequences=False),
          tf.keras.layers.Dense(1),
      ], name="vanilla_rnn_one_to_one")

      model_one_to_one.summary()
      show_model_diagram(model_one_to_one, "vanilla_rnn_one_to_one.png")

      X shape: (2099, 1, 5) y shape: (2099, 1)
```

Model: "vanilla_rnn_one_to_one"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 1, 32)	1,216
simple_rnn_1 (SimpleRNN)	(None, 32)	2,080
dense (Dense)	(None, 1)	33

- Add another **LSTM layer** to the LSTM model.

```
[15]: X1, y1 = make_one_to_one(train_scaled)
Xv1, yv1 = make_one_to_one(val_scaled)
print("X shape:", X1.shape, " y shape:", y1.shape)

model_one_to_one = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1, N_FEATURES)),
    tf.keras.layers.LSTM(UNITS, return_sequences=True),
    tf.keras.layers.LSTM(UNITS, return_sequences=False),
    tf.keras.layers.Dense(1),
], name="lstm_one_to_one")

model_one_to_one.summary()
show_model_diagram(model_one_to_one, "lstm_one_to_one.png")

X shape: (2099, 1, 5) y shape: (2099, 1)
```

Model: "lstm_one_to_one"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 1, 32)	4,864
lstm_1 (LSTM)	(None, 32)	8,320
dense (Dense)	(None, 1)	33

- Add another **GRU layer** to the GRU model.

```
[15]: X1, y1 = make_one_to_one(train_scaled)
Xv1, yv1 = make_one_to_one(val_scaled)
print("X shape:", X1.shape, " y shape:", y1.shape)

model_one_to_one = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1, N_FEATURES)),
    tf.keras.layers.GRU(UNITS, return_sequences=True),
    tf.keras.layers.GRU(UNITS, return_sequences=False),
    tf.keras.layers.Dense(1),
], name="gru_one_to_one")

model_one_to_one.summary()
show_model_diagram(model_one_to_one, "gru_one_to_one.png")

X shape: (2099, 1, 5) y shape: (2099, 1)
```

Model: "gru_one_to_one"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 1, 32)	3,744
gru_1 (GRU)	(None, 32)	6,336
dense (Dense)	(None, 1)	33

The additional layer should have the **same number of neurons** as the existing layer.

2. **Obtain all performance metrics** for each typology of each model (RNN, LSTM, and GRU):
 - MAE (Mean Absolute Error)
 - MSE (Mean Squared Error)
 - RMSE (Root Mean Squared Error)
 - R² Score (R-squared)

Vanilla RNN

Validation metrics (scaled target; lower error and higher R² are better):

	Model	Typology	MAE	MSE	RMSE	R ²
0	SimpleRNN	one_to_one	0.073914	0.008633	0.092916	0.722791
1	SimpleRNN	many_to_one	0.075278	0.008694	0.093241	0.723854
2	SimpleRNN	one_to_many	0.081213	0.010621	0.103060	0.658540
3	SimpleRNN	many_to_many_synced	0.072897	0.008390	0.091595	0.733641
4	SimpleRNN	many_to_many_seq2seq	0.076888	0.009171	0.095767	0.708759

LSTM

Validation metrics (scaled target; lower error and higher R² are better):

	Model	Typology	MAE	MSE	RMSE	R ²
0	LSTM	one_to_one	0.073319	0.008596	0.092713	0.724002
1	LSTM	many_to_one	0.071693	0.008093	0.089958	0.742953
2	LSTM	one_to_many	0.081235	0.010570	0.102811	0.660192
3	LSTM	many_to_many_synced	0.072754	0.008306	0.091139	0.736289
4	LSTM	many_to_many_seq2seq	0.071022	0.007981	0.089336	0.746560

GRU

Validation metrics (scaled target; lower error and higher R² are better):

	Model	Typology	MAE	MSE	RMSE	R ²
0	GRU	one_to_one	0.074284	0.008717	0.093367	0.720098
1	GRU	many_to_one	0.073535	0.008340	0.091325	0.735083
2	GRU	one_to_many	0.081041	0.010524	0.102588	0.661662
3	GRU	many_to_many_synced	0.072706	0.008278	0.090983	0.737191
4	GRU	many_to_many_seq2seq	0.071040	0.007915	0.088969	0.748639

3. **Determine which typology performs best** after adding another layer of the corresponding type (RNN, LSTM, or GRU). Discuss the results based on the performance metrics.

Answer:

Vanilla RNN

For the SimpleRNN model, the best result was achieved for Many-to-Many Synced type, which had the lowest values of MAE (0.072897), MSE (0.008390), and RMSE (0.091595) and the highest value of R^2 Score (0.733641). For the same model, the worst results were obtained for One-to-Many type with MAE equal to 0.081213 and R^2 Score equal to 0.658540. The addition of a second recurrent layer allowed the model to learn more complex sequential dependencies. However, the comparison of the results for Different Types shows that the choice of the typology is still important for the accuracy of predictions.

LSTM

For the long short-term memory (LSTM) model, the Many-to-Many Seq2Seq typology appeared to be the best-performing one, as it produced the lowest mean absolute error (MAE), namely, 0.071022, mean squared error (MSE): 0.007981, and root mean squared error (RMSE): 0.089336, and the highest R^2 Score: 0.746560. This can be explained by the fact that this type of recurrent neural network (RNN) architecture is particularly efficient in case of sequential data, including time series forecasting tasks, because of utilizing two LSTM layers for encoding and decoding stages. In contrast, the worst performer was the One-to-Many typology with MAE equal to 0.081235 and R^2 Score of 0.660192.

GRU

For the GRU model, the many-to-many Seq2Seq typology also performed the best. It had an MAE of 0.071040, MSE of 0.007915, RMSE of 0.088969, and R^2 Score of 0.748639. This implies that the stacked GRU Seq2Seq model was the most accurate model among all GRU typologies, as it predicted the required sequence accurately to determine the electricity consumption. Similar to the simple RNN and LSTM models, the one-to-many GRU typology also performed the worst with an MAE of 0.081041 and R^2 Score of 0.661662.

4. Advanced Challenge: Add one CNN layer before the RNN layer and run the model for 100 epochs using the following settings:

- Number of filters: 32
- Filter/kernel size: 3

```
[16]: X1, y1 = make_one_to_one(train_scaled)
Xv1, yv1 = make_one_to_one(val_scaled)
print("X shape:", X1.shape, " y shape:", y1.shape)

model_one_to_one = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1, N_FEATURES)),
    tf.keras.layers.Conv1D(filters=32, kernel_size=3, padding="same", activation="relu"),
    tf.keras.layers.SimpleRNN(UNITS, return_sequences=True),
    tf.keras.layers.SimpleRNN(UNITS, return_sequences=False),
    tf.keras.layers.Dense(1),
], name="vanilla_rnn_one_to_one")

model_one_to_one.summary()
show_model_diagram(model_one_to_one, "vanilla_rnn_one_to_one.png")

X shape: (2099, 1, 5) y shape: (2099, 1)

Model: "vanilla_rnn_one_to_one"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 1, 32)	512
simple_rnn (SimpleRNN)	(None, 1, 32)	2,080
simple_rnn_1 (SimpleRNN)	(None, 32)	2,080
dense (Dense)	(None, 1)	33

5. Provide a conclusion regarding the performance of the model using the settings in Item 4.

Validation metrics (scaled target; lower error and higher R^2 are better):

	Model	Typology	MAE	MSE	RMSE	R^2
0	SimpleRNN	one_to_one	0.075060	0.008684	0.093187	0.721173
1	SimpleRNN	many_to_one	0.076254	0.009257	0.096215	0.705953
2	SimpleRNN	one_to_many	0.080235	0.010375	0.101856	0.666474
3	SimpleRNN	many_to_many_synced	0.015236	0.000553	0.023509	0.982454
4	SimpleRNN	many_to_many_seq2seq	0.072513	0.008230	0.090720	0.738649

Conclusion:

Based on the experimental results after adding one CNN layer with 32 filters and kernel size of 3 before RNN layer and training the model for 100 epochs, the Many-to-Many Synced typology is the best among the five tested typologies. This model recorded the best performance with the lowest MAE (0.015236), MSE (0.000553) and RMSE (0.023509) with the highest R^2 Score (0.982454), indicating that the model had a very accurate prediction having captured almost all the data variance (explaining 98.25% of the variance in the scaled validation target). Compared with the previous SimpleRNN Many-to-Many Synced model without CNN layer which recorded an MAE, MSE, RMSE and R^2 Score of 0.072897, 0.008390, 0.091595 and 0.733641 respectively, the CNN–RNN architecture is significantly better with a large reduction in the model errors (MAE reduced by 79.31%, MSE reduced by 91.09%, RMSE reduced by 78.16% and finally, the R^2 improved by 32.65%).

While the other CNN–RNN typologies recorded smaller variations from their previous RNN counterparts, the Many-to-Many Synced model variation appears to have had the best improvements. The results demonstrate that adding a CNN layer to process local time features prior to stacking RNN layers could improve forecasting performance. All in all, the best-performing architecture among the five tested CNN–RNN typologies is the many-to-many synced configuration. However, further experimentation with different random seeds and an independent test set should be performed to further validate these results and confirm the reproducibility of such results.

6. **Compare the performance** of the model in Item 4 with the **one-layer RNN model** for each typology. Discuss and explain your findings.

Answer:

The CNN–RNN model that performed best used the Many-to-Many Synced typology. It had an R^2 Score of 0.982454 compared to the other CNN–RNN models, and it had significantly lower prediction errors than the others as well. This means that when forecasting in this experiment, using convolutional features alongside recurrent features proved to be the most beneficial. The convolutional part probably extracted relevant local patterns from the sequence before they were passed to the recurrent part.

However, it is notable that the performance gain was not consistent across all five types. The One-to-One, Many-to-One, One-to-Many, and Many-to-Many Seq2Seq designs had comparably much higher prediction errors. This means that adding a CNN layer does not necessarily improve the forecasting results; it depends on the input sequence design and the type of forecasting as well.